

Hierarchical planning

Main problem in **STRIPS**-like planning (as well as in other planning frameworks): complexity

One reason for complexity: no structure

- no distinction between important and unimportant properties
- no distinction between important and unimportant operators

This observation gives rise to two different ways of abstraction in

planning:

- abstraction of situations and
- abstraction of operators

Example : One level planner

Planning for "Going to Goa this Christmas"

- Switch on computer
- Start web browser
- Open Indian Railways website
- Select date
- Select class
- Select train.....

Practical problems are too complex to be solved at one level

Hierarchy of actions

- In terms of major action or minor action
- Lower level activities would detail more precise steps for accomplishing the higher level tasks.

Planning for "Going to Goa this Cristmas"

Major Steps :

- Hotel Booking
- Ticket Booking
- Reaching Goa
- Staying and enjoying there
- Coming Back

Minor Steps :

- Take a taxi to reach station / airport
- Have candle light dinner on beach
- Take photos

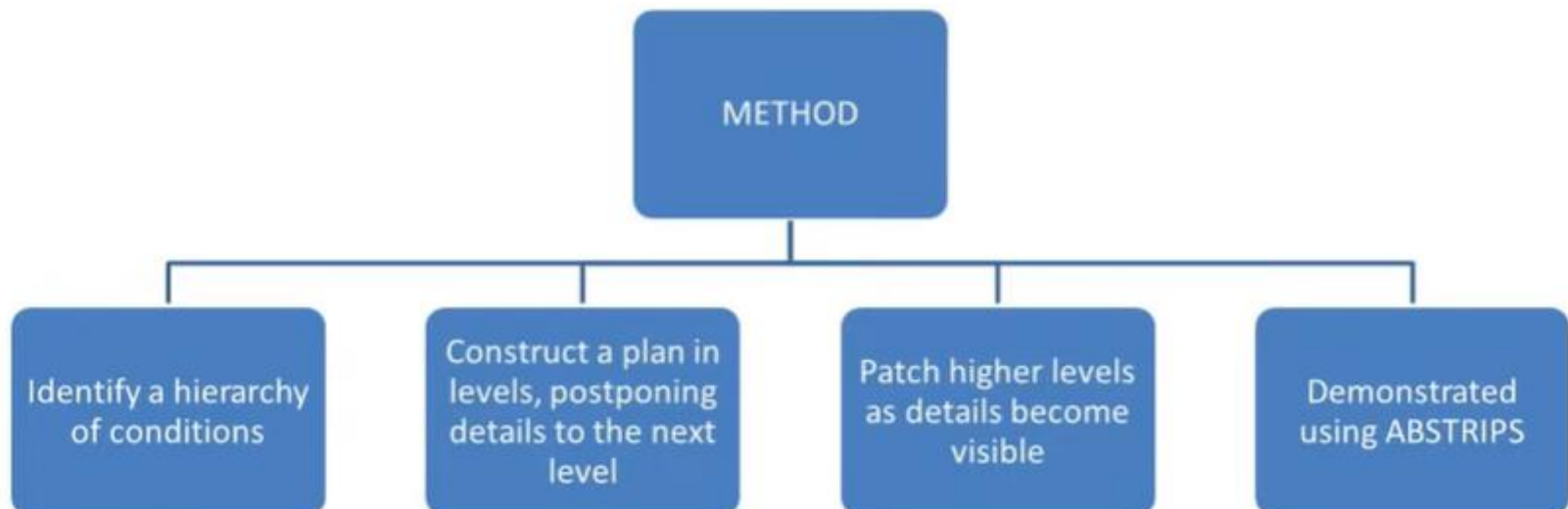
Reduces the size of search space

- Instead of having to try out a large number of possible plan ordering, plan hierarchies limit the ways in which an agent can select and order its primitive operators

If entire plan has to be synthesized at the level of most detailed actions, it would be impossibly long.

Postpone attempts to solve mere details, until major steps are in place.

Higher level plan may run into difficulties at a lower level, causing the need to return to higher level again to produce appropriately ordered sequence.



ABSTRIPS

Abstraction-Based STRIPS

Modified version of STRIPS that incorporates hierarchical planning

HERE , Hierarchy of conditions reflect the intrinsic difficulty of achieving various conditions.

Indicated by criticality value

A operation having minimum criticality can be trivially achievable, i.e., the operations having very less or no precondition.

Similarly operation having many preconditions to satisfy will have higher criticality.

Actions required for “Travelling to Goa”:

1. Opening makemytrip.com (1)
2. Finding flight (2)
3. Buy Ticket (3)
4. Get taxi(2)
5. Reach airport(3)
6. Pay-driver(1)
7. Check in(1)
8. Boarding plane(2)
9. Reach Goa(3)

1st level Plan :

- Buy Ticket (3),
Reach airport(3),
Reach Goa(3)

2nd level Plan :

- Finding flight (2),
Buy Ticket (3),
Get taxi(2),
Reach airport(3),
Boarding
plane(2), Reach
Goa(3)

3rd level Plan (final) :

- Opening
makemytrip.com
(1), Finding flight
(2), Buy Ticket (3),
Get taxi(2),
Reach airport(3),
Pay-driver(1),
Check in(1),
Boarding
plane(2), Reach
Goa(3)

OTHER PLANNING TECHNIQUES

1. Reactive systems
 - to avoid planning altogether.
2. Triangle tables
 - Record goal that each operator is expected to satisfy as well as the rules for correct execution
3. Meta planning
 - Reasoning not just problem but also about the planning process
4. Macro operators
 - Build new operators to represent the commonly used one
5. Case based planning
 - Reuse the old plans to make new ones

3.4.3 Planning and Acting in NonDeterministic World...

- ✓ Previously, we considered only **classical planning** domains that are **fully observable, static, and deterministic**.
- ✓ Furthermore, it is assumed that the action descriptions are **correct and complete**.
- ✓ In these circumstances, an agent can **plan first** and **then execute** the plan **"with its eyes closed."**

- ✓ But in an **uncertain environment**, on the other hand, an agent must **use its percepts** to discover what is happening.
- ✓ While the plan is being executed and possibly **modify or replace** the plan if something **unexpected happens**.
- ✓ Agents have to deal with both **incomplete** and **incorrect** information.
- ✓ **Incompleteness arises** because the world is partially observable, nondeterministic, or both.
- ✓ For example, I might believe that my key opens the supply cabinet, but **I could be wrong if the locks have been changed**.
- ✓ Without the ability to handle **incorrect information**, an agent can end up being as **unintelligent**.

- ✓ The possibility of having **complete or correct** knowledge depends on how much **indeterminacy** there is in the world.
- ✓ With **bounded indeterminacy**, actions can have unpredictable effects, but the possible effects can be listed in the action description axioms.
- ✓ For example, **when we flip a coin**, it is reasonable to say that the outcome will be **Heads or Tails**.
- ✓ With **unbounded indeterminacy**, on the other hand, the **set of possible preconditions or effects** either is **unknown** or is **too large** to be enumerated completely.
- ✓ This would be the case in very complex or dynamic domains such as **driving, economic planning**, and **military** strategy.

3.4.3.1 Planning methods in indeterminacy...

- ✓ There are **four planning methods** for handling indeterminacy.
- ✓ The **first two** are suitable for **bounded indeterminacy**, and the **second two** for **unbounded indeterminacy**:
 - i. **Sensorless Planning** - conformant planning
 - ii. **Conditional Planning** - contingency
 - iii. **Execution monitoring and replanning**
 - iv. **Continuous planning.**

i. Sensorless planning...

- ✓ Also called **conformant planning**, this method constructs standard, sequential plans that are to be executed **without perception**.
- ✓ The sensorless planning algorithm must ensure that the plan achieves the goal in all possible circumstances, **regardless of the true initial state** and the actual action outcomes.
- ✓ Sensorless planning relies on **coercion(force/demand)**-the idea that the world can be forced into a given state even when the agent has only partial information about the current state.
- ✓ **Coercion is not always possible**, so sensorless planning is often **inapplicable**.

ii. Conditional Planning...

- ✓ Also known as **contingency** planning, this approach deals with **bounded indeterminacy** by constructing a conditional plan with different branches for the different contingencies that could arise.
- ✓ Just as in classical planning, the agent plans first and then executes the plan that was produced.
- ✓ The agent finds out **which part of the plan** to execute by including sensing actions in the plan to test for the **appropriate conditions**.

iii. Execution monitoring and replanning...

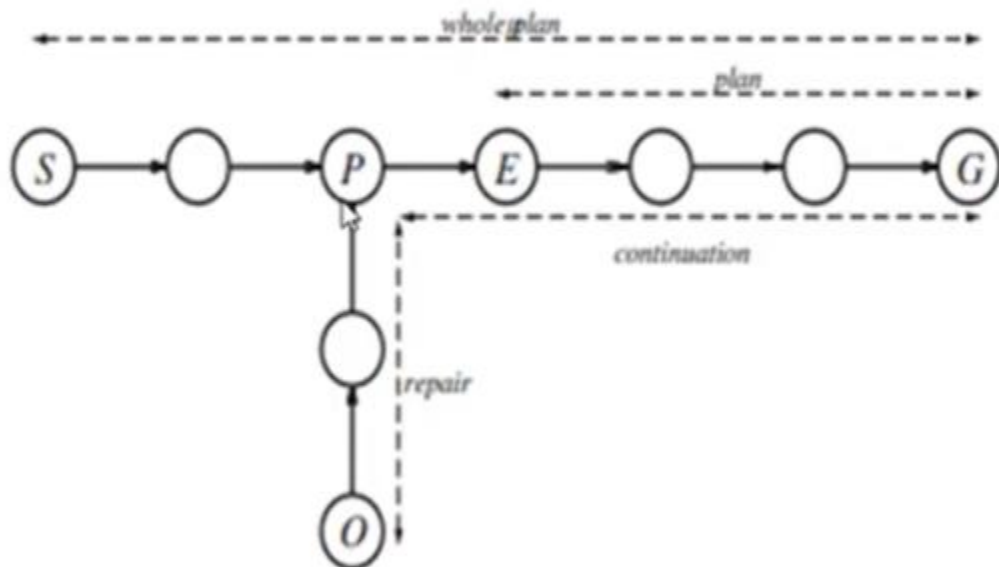


Figure 11.9 Before execution, the planner comes up with a plan, here called *whole plan*, to get from S to G . The agent executes steps of the plan until it expects to be in state E , but observes it is actually in O . The agent then replans for the minimal repair plus continuation to reach G .

Activate Wi
Go to Settings

iv Continuous planning...

- ✓ All the planners we have seen so far are designed to achieve a goal and then stop.
- ✓ A continuous planner is designed **to persist over a lifetime.**
- ✓ It can **handle unexpected circumstances** in the environment, even if these occur while the agent is in the middle of constructing a plan.
- ✓ It can also handle the **abandonment of goals** and the creation of additional goals by goal formulation.

Go to Settings to
Create Winx

PAINT EXAMPLE...



CONT...

- ✓ The problem is this: given an initial state with a **chair, a table, and some cans of paint**, with everything of unknown color, achieve the state where the chair and table have the same color.
- ✓ The differences among the various kinds of agents are:

A classical planning agent could not handle this problem, because the initial state is not fully specified—we **don't know what color the furniture is**.

Activate Windows
Go to Settings to activate Windows.

Solution...

Action(RemoveLid(can),

PRECOND: *Can(can)*

EFFECT: *Open(can)*)

Action(Paint(x, can),

PRECOND: *Object(x) $\wedge$ Can(can) $\wedge$ Color(can, c) $\wedge$ Open(can)*

EFFECT: *Color(x, c)*)

Percept(Color(x, c),

PRECOND: *Object(x) $\wedge$ InView(x)*

Percept(Color(can, c),

PRECOND: *Can(can) $\wedge$ InView(can) $\wedge$ Open(can)*

Activate
Go to Settings

i. Sensorless planning

✓ A **sensorless planning** agent must find a plan that works without requiring any sensors during plan execution.

➤ **Solution:** To open any can of paint and apply it to **both chair and table**, thus coercing them to be the same color.

[RemoveLid(Can_1), Paint($Chair$, Can_1), Paint($Table$, Can_1)] .

ii. Conditional planning...

- ✓ A conditional planning agent can generate a better plan:
 - i. First **sense the color** of the table and chair;
 - ii. If they are **already the same** then the plan is done.
 - iii. If not, **sense the labels** on the paint cans;
 - iv. If there is a can that is the same color as **one piece of furniture**, then apply the paint to the other piece.

Solution...

```
[LookAt( Table), LookAt( Chair),  
  if Color( Table, c)  $\wedge$  Color( Chair, c) then NoOp  
    else [RemoveLid( Can1), LookAt( Can1), RemoveLid( Can2), LookAt( Can2),  
      if Color( Table, c)  $\wedge$  Color( can, c) then Paint( Chair, can)  
      else if Color( Chair, c)  $\wedge$  Color( can, c) then Paint( Table, can)  
      else [Paint( Chair, Can1), Paint( Table, Can1)]]]
```

iii. Re-planning...

- ✓ A re-planning agent could generate the same plan as the conditional planner, or it could generate fewer branches at first and fill in the others at execution time as needed.
- ✓ It could also deal with incorrectness of its action descriptions.
- ✓ For example, suppose that the **Paint(obj, color)** action is believed to have the deterministic effect **Color (obj, color)**.
- ✓ A conditional planner would just assume that the effect has occurred once the action has been executed.
- ✓ But a re-planning agent would check for the effect, and **if it were not true** (perhaps because the agent was careless and missed a spot), it could then **replan to repaint the spot**.

Solution...

```
[LookAt( Table), LookAt( Chair),  
  if Color( Table, c)  $\wedge$  Color( Chair, c) then NoOp  
  else [RemoveLid( Can1), LookAt( Can1),  
        if Color( Table, c)  $\wedge$  Color( Can1, c) then Paint( Chair, Can1)  
        else REPLAN]] .
```

iv. Continuous Planning...

- ✓ A continuous planning agent, in addition to **handling unexpected events, can revise** its plans appropriately.
- ✓ **For example,** we can add the goal of having **dinner on the table**, so that the painting plan must be postponed.